

Caching

Strategies — Complete Study Notes

Cache Aside · Read Through · Write Around
Write Through · Write Back · Distributed Caching

TOPICS COVERED

1. What is Caching & Cache Hit/Miss

2. Distributed Caching & Consistent Hashing

3. Cache Aside Strategy

4. Read Through Cache Strategy

5. Write Around Cache

6. Write Through Cache

7. Write Back (Write Behind) Cache

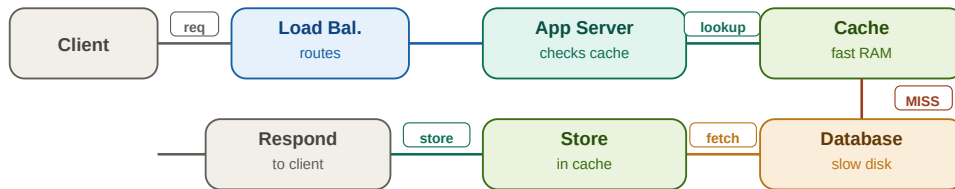
8. Quick Revision & Interview Questions

1. What is Caching?	3
2. Cache Hit & Miss	3
3. Distributed Caching & Consistent Hashing	3
4. Cache Aside (Lazy Loading)	4
5. Read Through Cache	4
6. Write Around Cache	5
7. Write Through Cache	5
8. Write Back (Write Behind) Cache	5
9. Strategy Comparison Table	6
10. Quick Revision & Interview Questions	6

1, 2 & 3. Caching Fundamentals & Distributed Caching

Caching stores frequently accessed data in fast memory (RAM) rather than fetching it from slow storage (disk/DB) every time. A cache hit returns data instantly; a cache miss goes to the DB, stores the result, and returns it.

What is Caching? — Flow, Hit/Miss Distributed Caching



HIT: Data found in cache → returned immediately. No DB query needed.

MISS: Not in cache → fetch DB → store → return to client.

Benefits: Lower latency · Less DB load · Higher throughput

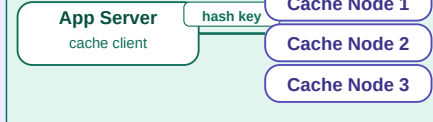
Cache tiers: Browser → CDN → Load Balancer → App Server

Distributed Caching — Why How

Single Cache Problems

- ✗ Limited by one server RAM
- ✗ Single point of failure
- ✗ Cannot scale horizontally

Cache Pool



Consistent Hashing

- Ring-based key→node mapping
- Even load distribution
- Add/remove: only ~1/N remapped
- Used by Redis Cluster, Memcached

Cache flow: App Server checks cache first · Hit = instant return · Miss = DB fetch → store in cache · Distributed cache pool via consistent hashing

Cache locations: Browser → CDN → Load Balancer → App Server (focus). Each layer reduces load on the next.

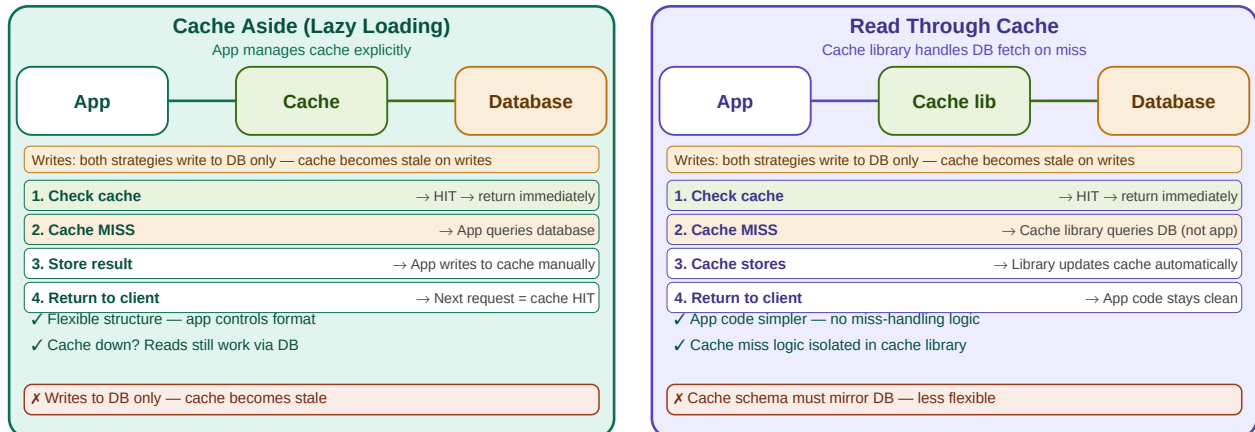
Consistent hashing: ring-based mapping of keys to cache nodes. Adding/removing nodes only remaps a fraction of keys — no full rehash.

Distributed cache (Redis cluster, Memcached): each app server has a cache client that hashes keys to the right cache node.

4 & 5. Read Strategies — Cache Aside & Read Through

Both read strategies handle cache misses but differ in **who is responsible** for fetching data from the DB on a miss — the application code or the cache library itself.

Read Caching Strategies — Cache Aside vs Read Through



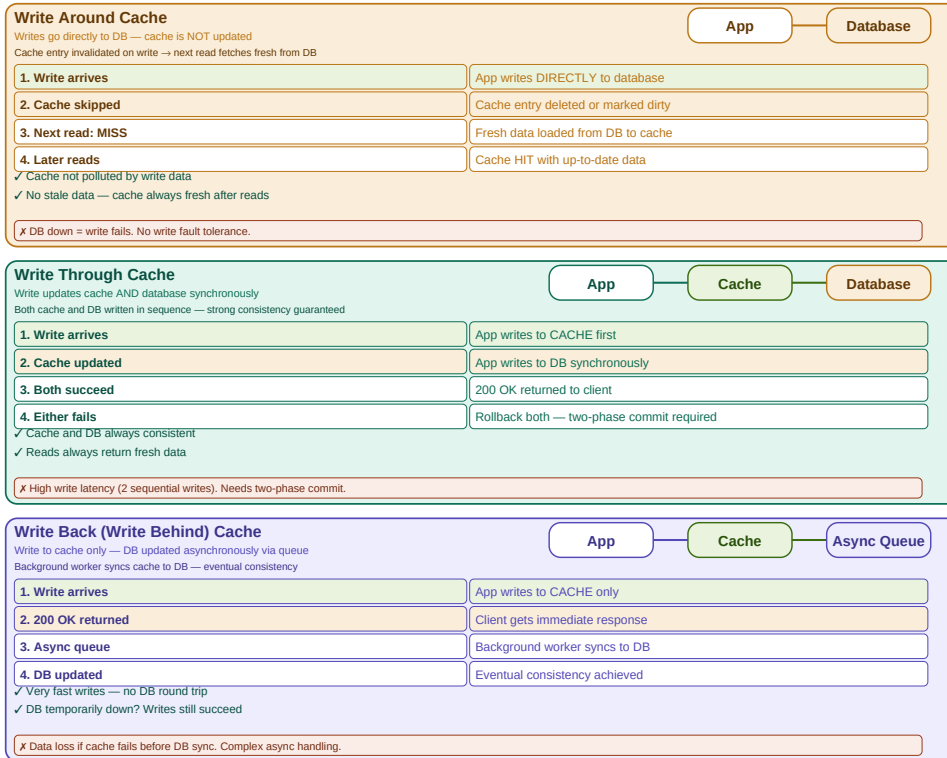
Cache Aside: app manages miss logic · Read Through: cache library handles miss automatically · Both: writes go to DB only

Aspect	Cache Aside	Read Through
Who fetches on miss	Application code queries DB and writes to cache	Cache library queries DB and stores automatically
App code complexity	More — miss logic in app code	Less — cache handles it all
Cache data structure	Flexible — app defines format	Must mirror DB schema exactly
Fault tolerance on read	If cache is down, reads still work via DB	Depends on cache library behaviour
Stale data risk	Yes — writes go to DB, cache not updated	Yes — same issue on writes

6, 7 & 8. Write Strategies — Around / Through / Back

Write strategies address the core problem of read strategies: **cache becoming stale when data is written**. Each approaches the cache-DB consistency problem differently.

Write Caching Strategies — Around / Through / Back



Write Around: DB only, cache invalidated · Write Through: cache+DB synchronous · Write Back: cache only, DB async

Strategy	Write path	Consistency	Fault tolerance	Write latency
Write Around	DB only; cache invalidated	Good — stale cache cleared	None — DB down = write fails	Medium (DB round trip)
Write Through	Cache then DB (sync)	Strong — always consistent	None — either down = write fails	High (cache + DB sequential)
Write Back	Cache only; DB async	Eventual consistency	Good — DB down = write succeeds	Low (cache only, fast)

9. Complete Strategy Comparison

All Caching Strategies — Quick Reference

Strategy	Type	How	Pros	Cons	Best for
Cache Aside	Read	App: check→miss→fetch DB	Simple; fault-tolerant reads	Cache stale on writes	read-heavy
Read Through	Read	Cache: miss→fetch DB auto	Clean app code	Schema mirrors DB	read-heavy
Write Around	Write	Write→DB; cache invalidated	No write pollution	No write fault tolerance	read+writes
Write Through	Write	Write→cache+DB sync	Always consistent	High latency; 2-phase	strong consistency
Write Back	Write	Write→cache; DB async	Fast writes; DB-down ok	Data loss if cache fails	write-heavy

5 strategies: type, how it works, pros, cons, best for

10. Quick Revision & Interview Questions

Quick Revision

- > Caching = storing frequently used data in RAM for fast access. Reduces DB load and latency.
- > Cache HIT: data found in cache → return immediately. Cache MISS: fetch from DB → store → return.
- > Distributed cache: multiple cache nodes. Each app server has a cache client using consistent hashing.
- > Consistent hashing: ring-based. Adding/removing nodes remaps only fraction of keys.
- > Cache Aside: app checks cache; on miss, app fetches DB and updates cache. Writes go to DB only.
- > Read Through: same as Cache Aside but cache library handles the DB fetch automatically on miss.
- > Cache Aside = flexible structure. Read Through = must mirror DB schema.
- > Both read strategies: writes go to DB only → cache can become stale after writes.
- > Write Around: writes go to DB, cache invalidated. Next read gets fresh data. No write fault tolerance.
- > Write Through: write to cache THEN DB synchronously. Always consistent. High write latency. Needs 2-phase commit.
- > Write Back: write to cache only. DB updated async via queue. Fast writes. Risk: data loss if cache dies.
- > Two-phase commit: needed in Write Through to rollback both cache and DB if either write fails.
- > TTL (Time to Live): expiry time for cached data. Important in Write Back — cache may expire before DB write.
- > No single strategy fits all — combine based on read-heavy vs write-heavy and consistency needs.
- > Read-heavy apps: Cache Aside or Read Through. Write-heavy: Write Back. Needs consistency: Write Through.

Interview Questions

1. What is caching? Where can it be placed in a system?
2. What is the difference between a cache hit and a cache miss?
3. What is distributed caching? How does consistent hashing help?
4. Explain Cache Aside. What are its advantages and drawbacks?
5. How does Read Through differ from Cache Aside?
6. What problem does Write Around Cache solve?
7. Explain Write Through Cache. Why does it need a two-phase commit?
8. What is Write Back Cache? When would you use it?

- 9. Which caching strategy has the best write fault tolerance? Why?
- 10. Which strategy has the worst write latency? Why?
- 11. A payment system needs strong consistency — which strategy would you choose?
- 12. A social media feed is read-heavy — which strategy fits best?
- 13. What is TTL and why is it especially important in Write Back caching?

Key Terms

Cache Hit	Data found in cache → returned immediately without DB query
Cache Miss	Data not in cache → must fetch from DB, then store in cache
Cache Aside	App manages cache: checks → miss → app fetches DB → app writes cache
Read Through	Cache manages miss: checks → miss → cache fetches DB → cache stores
Write Around	Write → DB only. Cache invalidated. Next read re-populates cache.
Write Through	Write → cache AND DB synchronously. Strong consistency. High latency.
Write Back	Write → cache only. DB updated async. Fast writes. Data loss risk.
Consistent Hashing	Ring-based hash mapping keys to cache nodes. Scales with node changes.
Dirty Flag	Marker that cached data is stale/invalid — must be re-fetched from DB
TTL	Time to Live — expiry time after which cached entry is purged
Two-phase Commit	Transaction pattern ensuring atomic write to both cache and DB
Distributed Cache	Cache pool across multiple nodes (Redis cluster, Memcached)
Eventual Consistency	Data converges to same state over time — may be briefly inconsistent
Cache Eviction	Process of removing old/unused entries when cache is full (covered in Part 2)